

IoT mikroservisi zasnovani na događajima

Uporedna evaluacija MQTT-a i Apache Kafke

Projekat 2 — Internet stvari i servisa

1. Opis sistema

Implementiran je asinhroni, event-driven mikroservisni sistem koji se sastoji od tri servisa kontejnerizovana pomoću Docker Compose-a:

- Data Ingestion Service (Node.js) — simulira 10 IoT uređaja koji čitaju podatke iz CSV dataseta i objavljuju poruke istovremeno na MQTT broker (Mosquitto) i Kafka topic (sensor-data) svakih ~100ms po uređaju. Kafka producer koristi device_id kao ključ poruke za konzistentno rutiranje na particije.
- Data Storage Service (Python/asyncpg) — pretplaćen na oba brokera, bufferuje poruke i upisuje ih batch-om od 500 poruka u PostgreSQL bazu podataka. Skaliran bez fiksnog container_name kako bi podržao više paralelnih instanci.
- Analytics Service (Node.js/KafkaJS) — pretplaćen na oba brokera, implementira Tumbling Window od 10 sekundi za agregaciju i detekciju anomalija. Meri end-to-end latenciju koristeći sent_at timestamp ugrađen u svaku poruku.

Kafka klaster koristi KRaft režim (bez Zookeeper-a) sa 3 brokera, replikacionim faktorom 3 i min.insync.replicas=2. Topic sensor-data ima 3 particije.

2. Arhitektura sistema

2.1 Kafka klaster — particije i replikacija

Topic sensor-data konfigurisan je sa 3 particije i replikacionim faktorom 3:

- Particija 0: Leader broker 1, replike na brokerima 1, 2, 3
- Particija 1: Leader broker 1, replike na brokerima 1, 2, 3
- Particija 2: Leader broker 2, replike na brokerima 2, 3, 1

Poruke se rutiraju po device_id ključu — sve poruke istog uređaja uvek idu na istu particiju, garantujući redosled po uređaju. Neravnomerna distribucija (Particija 1: 705 poruka, Particija 0: 1,666, Particija 2: 2,431) posledica je hešovanja 10 device_id vrednosti na 3 particije — savršena ravnomernost nije garantovana za mali broj ključeva.

2.2 Consumer Lag

Consumer lag predstavlja razliku između LOG-END-OFFSET i CURRENT-OFFSET po particiji. Pod normalnim opterećenjem lag je bio 3–30 poruka (zanemarljivo). Nakon 30-sekundnog prekida analytics kontejnera, lag je narastao na ukupno 10,214 poruka (particija 0: 5,339 | particija 1: 1,780 | particija 2: 3,095). Po ponovnom spajanju, Consumer Group se rebalansirao i sve poruke su replay-ovane za ~30 sekundi, lag se vratio na <20.

3. Eksperimentalni scenariji

3.1 Scenario A — Massive Sensor Ingestion

MQTT testiranje: emqtt-bench (zvanični alat) za sve tri QoS nivoe i tri skale klijenata.

Kafka testiranje: kafka-producer-perf-test.sh sa acks=all.

MQTT rezultati (emqtt-bench):

Klijenti	QoS	pub/s	overrun/s	pub_succ/s	Mosquitto RAM	Mosquitto CPU
100	0	~20,000	~2,000	N/A	4.6 MB	35%
100	1	~10,000	~5,500	~10,000	6.2 MB	44%
100	2	~8,000	~8,000	~8,000	6.1 MB	59%
1000	0	~22,000	~10,000	N/A	10.7 MB	21%
1000	1	~12,000	~12,000	~14,000	10.8 MB	75%
1000	2	~8,000	~8,000	~8,000	9.5 MB	61%
10000	0	~30,000	~15,000	N/A	10.1 MB	36%
10000	1	~20,000	~18,000	~20,000	12.2 MB	82%
10000	2	~8,000	~8,000	~8,000	7.7 MB	65%

* pub_overrun = poruke koje nisu isporučene na vreme. QoS 0 nema pub_succ jer nema potvrda isporuke.

Kafka rezultati (kafka-producer-perf-test.sh, acks=all):

Uređaji	Throughput	Avg lat.	p95 lat.	kafka1 CPU	kafka1 RAM
100	~100 msg/s	~130 ms	~450 ms	113%	843 MB
1000	~1,000 msg/s	~160 ms	~700 ms	78%	846 MB
10000 (max)	~8,954 msg/s	~15,320 ms	N/A	110%	1,142 MB

Ključne napomene:

- MQTT QoS 0 postiže najveći sirovi throughput (~30,000 pub/s, 10K klijenata) jer nema potvrda, ali overrun rate je gotovo jednak pub rate-u — većina poruka se gubi.
- MQTT QoS 2 (exactly once) ima najmanji throughput zbog dvostrukog handshake-a, ali Mosquitto RAM ostaje ispod 13 MB bez obzira na QoS nivo ili broj klijenata — izuzetno lagan za edge.
- Kafka sa acks=all dostiže ~9,000 msg/s ali troši 1.14 GB RAM-a samo na kafka1. Pod punim opterećenjem latencija eksplodira na 15+ sekundi.
- Storage servis se rušio pod visokim opterećenjem (0B/0B u docker stats) — I/O bottleneck PostgreSQL upisivanja, tačno kao što je navedeno u zadatku.

3.2 Scenario B — Edge Connectivity Failures

Testirani su prekidi i na strani producenta (ingestion) i na strani consumera (analytics).

Test	Broker	Ponašanje	Poruke izgubljene
Prekid producenta	MQTT	Windows prazni, konekcija automatski obnovljena	0 (poruke nisu ni slane)
Prekid producenta	Kafka	Lag ostao ~0, nema novih poruka	0
Prekid consumera	MQTT	Poruke odbačene — nema persistence	~1,200+ (15s × ~83/s)
Prekid consumera	Kafka	10,214 poruka u backlog-u, replay za ~30s	0

MQTT detalji:

- Subscriber (emqtt-bench sub) prima ~83–87 msg/s u normalnom radu. Nakon docker stop i ponovnog pokretanja, subscriber odmah prima isti rate novih poruka — nema zaostalih poruka iz perioda offline. Mosquitto ih je jednostavno odbacio jer emqtt-bench koristi cleanSession=true (default).
- Ingestion servis ima ugrađen reconnectPeriod: 1000ms — automatski se spaja bez gubitka novih poruka čim mreža postane dostupna.

Kafka detalji:

- Analytics consumer group po prekidu akumulirao je 10,214 poruka u backlog-u across 3 particije. Rebalans Consumer Group-a vidljiv u logovima: 'Consumer has joined the group' sa novim memberId.
- Replay je kompletiran za ~30 sekundi, lag vratio na <20. Nijedna poruka nije izgubljena zahvaljujući offset commit mehanizmu.

3.3 Scenario C — Burst Event Load

Simuliran nagli prelaz sa baseline load-a na maksimalni burst za oba brokera.

Broker	Faza	Throughput	Avg lat.	overrun/s	Ponašanje
MQTT	Baseline (50)	50 msg/s	<10 ms	0	Stabilno
MQTT	Burst (5000)	~9,000 pub/s	<10 ms	~9,000/s	Gubitak poruka, nema backlog-a
Kafka	Baseline (50)	48.6 msg/s	670 ms	0	Stabilno
Kafka	Burst (max)	~2,027 msg/s	10,228 ms	N/A	Leader election, retry, akumulacija

MQTT burst analiza:

- Mosquitto RAM ostaje ~16 MB čak i pod burstom od 5,000 klijenata — nema akumulacije poruka u memoriji jer Mosquitto nema backlog mehanizam.
- `pub_overrun` $\approx$ `pub` total tokom bursta — broker prima i odmah odbacuje poruke koje ne može dostaviti subscriberima. Ovo je MQTT backpressure: gubitak poruka umesto povećane latencije.

Kafka burst analiza:

- Tokom bursta pojavile su se `NOT_LEADER_OR_FOLLOWER` greške — kafka1 privremeno izgubio liderstvo nad particijama pod CPU pritiskom (149%). Klaster je automatski izvršio leader election i producent nastavio sa retry mehanizmom.
- P95 latencija dostigla 18,911 ms. Za razliku od MQTT-a, Kafka akumulira poruke u log i garantuje isporuku — cena je povećana latencija, a ne gubitak poruka.
- Recovery time (vreme do normalnog lag-a): ~60–90 sekundi nakon završetka bursta.

3.4 Scenario D — Real-Time Alerting Latencija

End-to-end latencija merena u Analytics servisu kao razlika između `sent_at` (timestamp generisanja u Ingestion servisu) i trenutka procesiranja. Merenja su sprovedena tokom normalnog rada sistema (10 uređaja, ~100ms interval).

Broker	Stanje	Avg lat.	Min lat.	Max lat.
MQTT	Stabilno	~4,970 ms*	1 ms	~10,010 ms
MQTT	Catchup nakon prekida	~625–8,360 ms	2 ms	~10,433 ms
Kafka	Stabilno	~5,100 ms*	4 ms	~10,460 ms
Kafka	Catchup (consumer off)	~14,000–25,000 ms	38 ms	~29,998 ms

* Prosečna latencija od ~5,000 ms je dominantno posledica Tumbling Window agregacije od 10 sekundi — poruka čeka do isteka prozora pre nego što se alarm generiše. Minimalna latencija (1 ms za MQTT, 4 ms za Kafka) predstavlja realni end-to-end signal za poruke primljene tik pre isteka prozora.

Razlika između brokera:

- MQTT min latencija od 1 ms potvrđuje da broker ne uvodi skoro nikakav overhead — poruka ide direktno od publishera do subscribera.
- Kafka min latencija od 4 ms odražava overhead replikacije (`acks=all`, `min.insync.replicas=2`) i batch procesiranja consumera.
- U stabilnom stanju, prosečne latencije su praktično identične jer ih dominira Window period, a ne overhead brokera.
- Tokom catchup-a nakon prekida, Kafka latencija skače na 14,000–25,000 ms jer consumer procesira stare poruke sa velikim `sent_at` offsetom.

4. Analiza pouzdanosti i garancija isporuke

4.1 Zašto je MQTT idealan za edge, a neadekvatan za historijsku analitiku?

Prednosti MQTT za edge:

- Ekstremno mali footprint: Mosquitto koristio 5–12 MB RAM-a čak sa 10,000 klijenata. Može se pokrenuti na Raspberry Pi, ESP32 ili industrijskom PLC-u.
- Minimalan protokolni overhead: MQTT fixed header svega 2 bajta. Idealno za senzore na bateriji ili GPRS/LTE vezama sa ograničenim bandwidthom.
- QoS fleksibilnost: QoS 0 za nekritične senzore (temperatura svake sekunde), QoS 1 za alarme, QoS 2 za komande aktuatorima — prilagodljivo po tipu podataka.
- Brzo spajanje/razdvajanje: Will poruke i persist sesije omogućavaju graceful disconnect za mobilne uređaje.

Ograničenja za historijsku analitiku:

- Nema trajne memorije poruka: Mosquitto ne čuva istoriju. Subscriber koji je offline gubi sve poruke (dokazano u Scenario B — ~1,200 poruka izgubljeno za 15 sekundi).
- Nema replay mehanizma: Ne postoji mogućnost ponovnog čitanja starih poruka — osnova za batch analitiku, ML treniranje i auditing.
- Nema skalabilnog particionisanja: Jednom porukom ne mogu se paralelno baviti više consumer instanci na strukturiran način.

4.2 Zašto Kafka dominira u cloud sistemima i kolika je cena?

Prednosti Kafke za cloud:

- Trajno čuvanje poruka na disku: Svaka poruka upisuje se u log sa konfigurisanim retention periodom. Više consumera može čitati isti topic nezavisno i u različito vreme.
- Horizontalna skalabilnost: Dodavanjem brokera i particija throughput se linearno povećava. Offset mehanizam garantuje crash recovery bez gubitka poruka (dokazano: 10,214 poruka replay-ovano).
- Consumer Groups i paralelno procesiranje: Više instanci storage servisa mogu simultano konzumirati različite particije — linearno skaliranje I/O kapaciteta.
- Fault tolerance: RF=3 i min.insync.replicas=2 garantuju da klaster preživljava pad jednog brokera bez gubitka podataka. Leader election automatski vraća sistem u normalu.

Cena skalabilnosti:

- Resursi: Svaki Kafka broker zahteva 600–1,142 MB RAM-a za JVM. Ukupno 3 brokera = ~2.1 GB RAM-a samo za Kafku — neprihvatljivo za ARM edge uređaje.
- Kompleksnost: Kafka zahteva klaster (minimum 3 brokera za RF=3), monitoring consumer lag-a, konfiguraciju particija i retention policy-ja.
- Latencija pod opterećenjem: Burst load izazvao latenciju do 18,911 ms (p95), leader election i recovery period od 60–90 sekundi.
- Edge nekompatibilnost: Čak i Kafka KRaft (bez Zookeeper-a) ostaje isključivo server/cloud tehnologija. 600 MB RAM minimum po brokeru je ~100x više od Mosquitto-a.

5. Uporedna tabela performansi

Metrika	MQTT (Mosquitto)	Apache Kafka (KRaft)
Maks. throughput	~30,000 pub/s (QoS 0, 10K klijenata)	~8,954 msg/s (acks=all, 3 brokera)
Avg alert latencija (stabilno)	~4,970 ms*	~5,100 ms*
Min alert latencija	1 ms	4 ms
Max alert latencija	~10,010 ms	~10,460 ms
p95 latencija (1K msg/s)	N/A	700 ms
p95 latencija (burst)	N/A (gubitak poruka)	18,911 ms
RAM footprint	5–12 MB (Mosquitto)	600–1,142 MB po brokeru
CPU (vršno, burst)	82% (10K klijenata, QoS 1)	149% (kafka1, burst)
Trajnost poruka	Ne (bez replay-a)	Da — offset replay
Recovery (consumer off 30s)	~1,200 poruka izgubljeno	10,214 poruka replay-ovano
Burst ponašanje	Gubitak poruka (overrun)	Akumulacija + leader election + retry
Garancija isporuke	QoS 0/1/2	acks=0/1/all
Edge deployment	Idealan (5–12 MB)	Neprikladan (600+ MB/broker)

* Prosečna latencija uključuje vreme čekanja u Tumbling Window od 10 sekundi. Minimalna latencija je realni end-to-end signal.

6. Zaključak

Svi eksperimenti su sprovedeni na realnom Docker Compose stack-u i potvrdili su teorijska svojstva oba sistema:

- MQTT je superioran za edge sloj: minimalan resursni otisak (5–12 MB), brze konekcije, fleksibilan QoS. Idealan za prvu liniju IoT komunikacije između senzora i lokalnog gateway-a.
- Kafka dominira u cloud data pipeline-ima: garantovana trajnost, offset replay, particionisana skalabilnost i consumer group paralelizam — uz cenu visokih resursa (~2.1 GB RAM za 3-brokerski klaster).
- Ključna razlika potvrđena eksperimentalno: Kafka je replay-ovala 10,214 poruka nakon 30-sekundnog prekida consumera; MQTT je izgubio ~1,200 poruka za isti period.
- Burst ponašanje se fundamentalno razlikuje: MQTT gubi poruke (overrun) bez povećanja latencije; Kafka akumulira poruke i povećava latenciju (do 18,911 ms p95) ali ne gubi podatke.
- Hibridna arhitektura (MQTT na edge → Gateway → Kafka u cloudu) je optimalno rešenje za IoT sisteme koji zahtevaju i laganu edge komunikaciju i pouzdanu cloud analitiku.